Задача 7: Методи регресійного аналізу в Python

Загальна мета полягає в тому, щоб пояснити ціни продажу `SalePrice` за допомогою набору змінних, використовуючи набір даних із попереднього домашнього завдання (Ames Housing). Змінна `SalePrice` приймається як залежна змінна (target), а решта змінних — як пояснювальні (features).

(1) Регресія ласо (Lasso) та рідж-регресія (Ridge) є методами регуляризації, які використовуються для стабілізації оцінювання параметрів.

   (а) Поясніть своїми словами концепцію регресії ласо. Опишіть сценарій, за якого звичайна лінійна регресія зазнає невдачі (наприклад, через мультиколінеарність або коли $p > n$), але регресію ласо все ще можна оцінити. Чому нам потрібно максимізувати цільову функцію чисельно (на відміну від рідж-регресії, яка має аналітичний розв'язок)?

   (б) Для звичайної регресії МНК змінні рідко нормуються. Однак у випадку ласо-регресії масштабування є критичним, оскільки штраф накладається безпосередньо на абсолютну величину коефіцієнтів. Чому це так? Масштабуйте числові дані за допомогою `StandardScaler`. Чи можна / чи потрібно масштабувати бінарні змінні (даммі) в такий самий спосіб? Як обробити факторні змінні (наприклад, `FireplaceQu`) через `OneHotEncoder`?

   (в) Запустіть регресію ласо за допомогою `Lasso` або `LassoCV` з коефіцієнтом регуляризації $\alpha > 0$ (α в `sklearn` відповідає λ в теоретичних формулах). Побудуйте графік залежності оцінених параметрів від α. Яке значення α ви б рекомендували? Визначте оптимальний α за допомогою 5-блочної крос-валідації (`LassoCV`).

   (г) Розгляньте рідж-регресію (`Ridge` або `RidgeCV`). Оцініть її для оптимального значення α, знайденого через крос-валідацію. Порівняйте оцінені параметри та їхні дисперсії для рідж-регресії та множинної лінійної регресії (`LinearRegression`). Чого ви очікуєте від такого порівняння з точки зору компромісу між зміщенням та дисперсією (bias-variance tradeoff)? Не забудьте підігнати лінійну регресію до попередньо масштабованих змінних.

(2) На наступному кроці ми моделюємо `SalePrice` за допомогою дерев регресії, використовуючи `DecisionTreeRegressor` з бібліотеки `sklearn.tree`.

   (а) Нехай `GrLivArea` буде першою змінною, яка використовується для розщеплення (splitting). Запишіть відповідну задачу оптимізації (мінімізацію суми квадратів залишків RSS для двох дочірніх вузлів) та поясніть, як працює алгоритм жадібного пошуку точки розщеплення.

   (б) *(опціонально)* Реалізуйте цю оптимізацію вручну на Python. Упорядкуйте значення `GrLivArea`. Створіть сітку (grid) для потенційних точок розщеплення так, щоб кожен елемент сітки знаходився посередині між двома послідовними спостереженнями `GrLivArea` (тобто $(x_{(i)}+x_{(i+1)})/2$). Обчисліть значення цільової функції (RSS) для кожного елемента сітки та побудуйте його графік за допомогою `matplotlib`. Порівняйте свої ручні результати з результатами `DecisionTreeRegressor` (встановіть `max_depth=1`).

(в) Тепер розгляньте змінну `OverallQual`. Вона приймає 10 різних упорядкованих значень (від 1 до 10). Як алгоритм дерева на Python визначає оптимальне розщеплення для такого типу дискретних або рангових змінних? Опишіть логіку обробки категоріальних та порядкових ознак у деревах.

(г) Очевидно, що без обмежень можна отримати дуже великі перенавчені дерева. Стрижка дерев (tree pruning) допомагає досягти розумного розміру. Підгоніть модель CART за допомогою `DecisionTreeRegressor` і обмежте її так, щоб вона мала не більше 10 розщеплень (параметр `max_leaf_nodes=11`).

(д) Знайдіть значення параметра складності (Cost-Complexity Pruning alpha, `ccp_alpha`) за допомогою методу `cost_complexity_pruning_path()` для дерева, навченого в попередньому пункті. Що це за параметр і що він регулює? Що станеться з архітектурою дерева, якщо ми суттєво збільшимо або зменшимо `ccp_alpha`? Перевірте це експериментально на ваших даних.

(е) Перевірте значення покращення (improvement / зменшення критерію неоднорідності) у першому розщепленні дерева. Поясніть концепцію інформаційного прибутку (impurity reduction) та наведіть числовий вираз, за яким воно обчислюється в `scikit-learn` для першого вузла.

(ж) Обчисліть важливість ознак (`feature_importances_`) для дерева регресії та порівняйте топ-10 результатів із коефіцієнтами, отриманими в регресії ласо та класичній лінійній регресії.

(з) *(опціонально)* Застосуйте техніку бутстрепу (bootstrap) для оцінки розподілу параметрів у лінійній регресії. Розгляньте лише перші 10 числових ознак. Створіть цикл на 1000 ітерацій: у кожній ітерації згенеруйте бутстреп-вибірку за допомогою `df.sample(replace=True)`, оцініть коефіцієнти через `LinearRegression` і збережіть значення для `LotArea` та `Street`. Побудуйте графік у вигляді гістограми або KDE (оцінку щільності ядра) через `seaborn.kdeplot` для цих двох параметрів. Порівняйте їх із теоретичним нормальним розподілом МНК (де дисперсія береться з діагоналі коваріаційної матриці залишків $\hat{\sigma}^2(X'X)^{-1}$).

(и) *(опціонально)* Використовуючи отриманий бутстреп-розподіл коефіцієнтів, обчисліть 95% довірчий інтервал через квантилі (`np.percentile` на рівнях 2.5% та 97.5%). Порівняйте їх із класичними довірчими інтервалами, що базуються на припущенні про нормальність залишків (які можна отримати через `summary().tables` в `statsmodels`).

(і) Використовуйте беггінг (`BaggingRegressor`) та випадковий ліс (`RandomForestRegressor`) для прогнозування `SalePrice`. Побудуйте графік залежності середньоквадратичної помилки (MSE) на навчальній та валідаційній вибірках від кількості базових дерев (`n_estimators`). Прокоментуйте поведінку обох кривих на графіку.

(к) Порівняйте час виконання (`time.time()`) обох ансамблевих підходів. Чому випадковий ліс зазвичай обчислювально швидший і потребує менше дерев порівняно з чистим беггінгом над глибокими деревами? Перевірте вбудований метод розрахунку важливості ознак для `RandomForestRegressor` (MDI або permutation importance). Зробіть фінальну порівняльну табли-

цю важливостей змінних для всіх моделей: Lasso, OLS, одиночне дерево та Random Forest.

(3) Перейдемо до задачі бінарної класифікації. Визначимо цільову змінну:

$$Yi = \begin{cases} 1, & \text{якщо } \texttt{SalePrice} > 130000 \\ 0, & \text{в іншому випадку.} \end{cases}$$

(а) Оцініть логістичну регресію, використовуючи покроковий зворотний відбір (Backward Selection) за інформаційним критерієм AIC. Для реалізації цього алгоритму та зручного розрахунку AIC використовуйте модуль `statsmodels.formula.api.logit` або `sm.Logit`. Щоб прискорити обчислення, почніть покроковий відбір лише з числових ознак (features). Надалі аналізуйте виключно фінальну підібрану модель.

(б) Розгляньте коефіцієнт біля змінної `LotArea`. Його не можна інтерпретувати лінійно. Наведіть правильну економічну інтерпретацію через відношення шансів (Odds Ratio), експоненціювавши коефіцієнт: $\exp(\beta_{\texttt{LotArea}})$.

(в) Сформуйте списки змінних, які статистично значуще збільшують ймовірність події $Y = 1$ (додатній коефіцієнт), та тих, що її зменшують (від'ємний коефіцієнт). Чи узгоджуються ці знаки з базовою економічною логікою ринку нерухомості?

(г) Візьміть випадкові 5 будинків із вибірки та розрахуйте для них ймовірність належати до класу 1 за допомогою методу `.predict()`. Запишіть повну математичну формулу логістичної функції $p = \frac{1}{1+e^{-z}}$ для першого будинку, підставивши туди реальні значення його ознак та оцінені моделлю коефіцієнти. Який найпростіший спосіб перейти від ймовірностей до безпосереднього прогнозу класу (класичний поріг cutoff)?

(д) Побудуйте матрицю помилок за допомогою функції `confusion_matrix` з `sklearn.metrics` для стандартного порогу 0.5. Розрахуйте чутливість (recall / sensitivity), специфічність та загальну точність (accuracy). Надайте розгорнуту словесну інтерпретацію кожної клітинки матриці помилок у контексті вартості та доступності будинків.

(е) Оскільки поріг 0.5 не завжди є оптимальним, побудуйте ROC-криву за допомогою `roc_curve` та знайдіть площу під нею (`roc_auc_score`). Визначте оптимальний поріг класифікації, максимізувавши індекс Юдена (Sensitivity+Specificity−1). Побудуйте графік кривої за допомогою `matplotlib`.

(ж) Перерахуйте матрицю помилок, чутливість та специфічність для нового (оптимального за ROC) порогу. Порівняйте ці метрики з отриманими при порозі 0.5. Дайте відповідь: чи стала ваша модель після зміни порогу більш «консервативною» щодо прогнозування дорогих будинків (клас 1)?